Đề cương: TinyML Accelerator trên FPGA

Oct 15, 2026 · @Viet

Đề bài gốc còn thiếu gì

Đề bài gốc mới nói *làm gì* và *ai làm*, chưa nói *làm tới đâu là đạt* và *hơn cái gì*. Cụ thể còn thiếu 7 điểm:

1. **Không có mục tiêu định lượng:** chưa nêu độ chính xác, độ trễ, tài nguyên, công suất cần đạt.
2. **Không có đối tượng so sánh:** "tăng tốc" phải nói rõ nhanh hơn cái gì (CPU, vi điều khiển, CPU mềm trên cùng FPGA, hoặc thiết kế FPGA đã công bố).
3. **Chưa chốt board và bài toán:** đề để "Basys 3 hoặc DE10-Lite", "MNIST hoặc keyword spotting". Đề cương này chọn Basys 3 + MNIST.
4. **Chưa nói dữ liệu vào/ra:** ảnh đưa vào board bằng gì, kết quả hiển thị ở đâu (ở đây là UART và LED 7 đoạn).
5. **Chưa có cách kiểm chứng:** làm sao biết phần cứng tính đúng (ở đây là so từng bit với mô hình Python).
6. **Chưa có quy ước giữa hai người:** định dạng file trọng số, cách lượng tử, cách làm tròn.
7. **Chưa nêu điểm mới của đề tài:** vì sao dùng FPGA mà không dùng vi điều khiển, và nhóm tự thiết kế khác gì so với dùng công cụ có sẵn như FINN hay hls4ml.

Mục tiêu đề tài

Mục tiêu tổng quát: tự thiết kế (không dùng HLS hay công cụ sinh tự động) một bộ tăng tốc int8 cho mạng nơ-ron trên FPGA giá rẻ, chạy nhanh hơn nhiều lần so với bộ xử lý nhúng cùng tầm giá, với độ chính xác gần như bằng mô hình float32.

Mã	Mục tiêu cụ thể	Mức cơ bản	Mức mở rộng
MT1	Độ chính xác int8 trên 10.000 ảnh test	≥ 95 %, mất ≤ 1 % so với float32	≥ 97,5 % (lớp ẩn lớn hơn, QAT)
MT2	Phần cứng khớp mô hình Python	100 % từng bit	—
MT3	Độ trễ mỗi ảnh @ 100 MHz	≤ 50 µs (P = 8)	≤ 10 µs (P = 32, bỏ qua pixel 0)
MT4	So với CPU mềm MicroBlaze trên cùng Basys 3	nhanh hơn ≥ 20 lần	nhanh hơn ≥ 100 lần

Mã	Mục tiêu cụ thể	Mức cơ bản	Mức mở rộng
MT5	Tài nguyên Basys 3	⩽ 50 % DSP và BRAM	bảng đánh đổi theo P
MT6	Năng lượng mỗi ảnh	có số đo từ Vivado Power Report	so với CPU và vi điều khiển
MT7	Demo	nhận dạng ảnh gửi từ PC qua UART	ảnh vẽ tay thật

Con số của MT4 dựa trên ước tính MicroBlaze ở mục so sánh bên dưới, cần đo thật ở tuần 6.

Phạm vi

Xây một bộ tăng tốc phần cứng trên FPGA Basys 3 nhận ảnh chữ số viết tay 28×28 qua UART và trả về chữ số dự đoán, chạy mạng nơ-ron int8 do nhóm tự huấn luyện.

- **Board:** Digilent Basys 3 (Artix-7 XC7A35T, Vivado). DE10-Lite là phương án dự phòng.
- **Bài toán:** MNIST với mạng MLP 784-32-10. Keyword spotting chỉ để làm phần mở rộng, vì cần thêm micro và khối MFCC.
- **Yêu cầu đầu ra:** độ chính xác int8 ⩾ 95 %, kết quả phần cứng khớp từng bit với mô hình Python, chạy được trên board.
- **Ngoài phạm vi:** bộ nhớ ngoài (DDR), CPU mềm, huấn luyện trên FPGA.

So sánh với các mạch và nền tảng có sẵn

Bản hiện tại (32 µs/ảnh) chậm hơn CPU máy tính (4,9 µs), nhưng ước tính nhanh hơn khoảng 60–80 lần so với CPU mềm trên cùng Basys 3. Vì vậy đối chứng chính của đề tài là bộ xử lý nhúng, còn CPU máy tính và các thiết kế FPGA đã công bố dùng để đặt đề tài vào bức tranh chung.

Nền tảng	Mô hình, độ chính xác	Độ trễ mỗi ảnh	Công suất	Tài nguyên	Loại số liệu
Đề tài, Basys 3 @ 100 MHz, P = 8	MLP int8 784-32-10, 96,57 %	32 µs	cần đo	~22 DSP, ~8 BRAM36	độ trễ đo bằng mô phỏng; tài nguyên ước tính
Đề tài, Basys 3 @ 100 MHz, P = 32	như trên	~9 µs	cần đo	~46 DSP	ước tính
CPU mềm MicroBlaze trên Basys 3 @ 100 MHz	cùng mô hình int8	~2–2,5 ms	—	—	ước tính (8–10 chu kỳ/MAC × 25.408 MAC)

Nền tảng	Mô hình, độ chính xác	Độ trễ mỗi ảnh	Công suất	Tài nguyên	Loại số liệu
Vi điều khiển ARM Cortex-M4 @ 84 MHz	cùng mô hình int8	~0,3–1,5 ms	—	—	ước tính (1–2 chu kỳ/MAC với CMSIS-NN, ~5 với C thường)
CPU Intel Xeon 2,8 GHz, C -O2, 1 luồng	cùng mô hình int8	4,9 µs	—	—	đo thật (`ai/cpu_baseline.py`)
FINN SFC-fix, ZC706 (Zynq Z7045) @ 200 MHz	MLP trọng số 1 bit, 95,83 %	240 µs	0,4 W (chip)	5.155 LUT, 16 BRAM	công bố
FINN SFC-max, ZC706 @ 200 MHz	MLP trọng số 1 bit, 95,83 %	0,31 µs (12,36 triệu ảnh/s)	7,3 W (chip)	91.131 LUT, 4,5 BRAM	công bố
SNN trên Basys 3 @ 100 MHz	mạng nơ-ron xung 784-128-10	520 µs	0,381 W	6.358 LUT, 40,5 BRAM	công bố

Nhận xét:

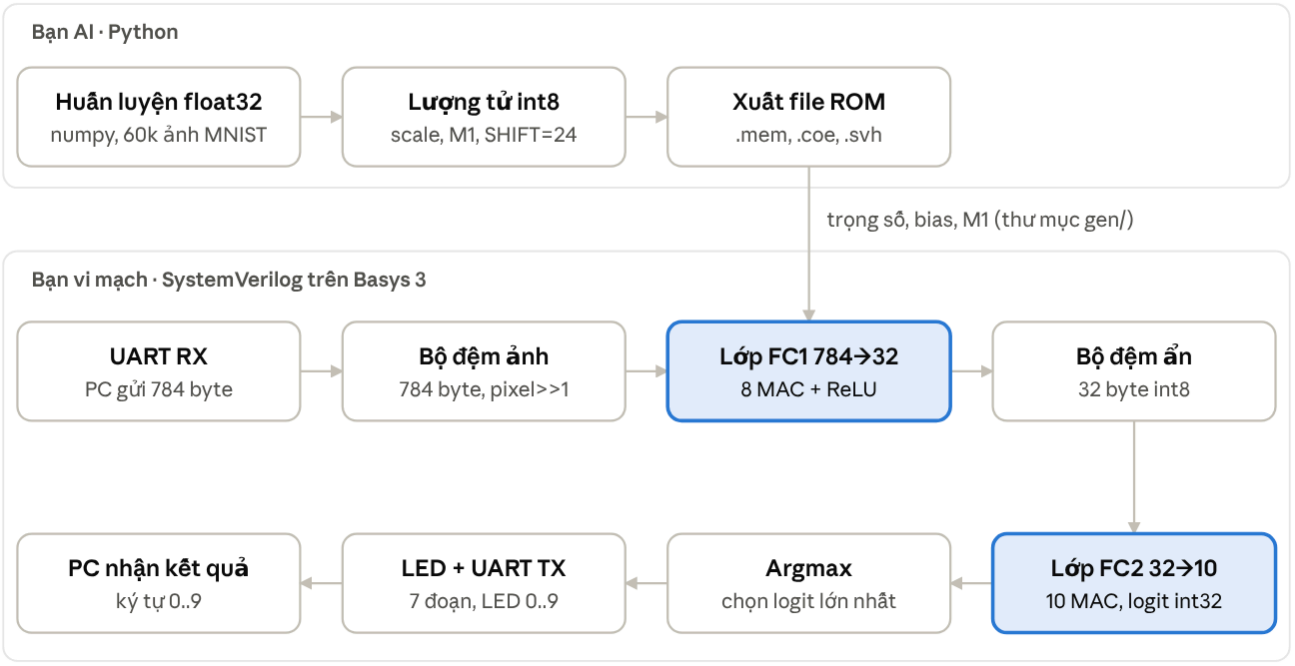
- **So với bộ xử lý nhúng** (MicroBlaze, Cortex-M4): đây là đối chứng chính, vì cùng tầm giá và cùng mô hình. Nhóm nên dựng MicroBlaze trên chính Basys 3 và đo thật (tuần 6), dùng lại file C do `ai/cpu_baseline.py` sinh ra.
- **So với CPU máy tính:** bản P = 8 thua về độ trễ. Bản P = 32 có bỏ qua pixel 0 có thể ngang, trong khi FPGA tiêu thụ công suất thấp hơn nhiều và có độ trễ cố định.
- **So với FINN:** FINN đạt tốc độ cực cao nhờ trọng số 1 bit và chip lớn. Riêng SFC-max dùng 91.131 LUT, gấp hơn 4 lần toàn bộ Basys 3 (20.800 LUT). Đề tài dùng int8 trên chip nhỏ, nên so với FINN để thấy đánh đổi giữa số bit, tốc độ và tài nguyên, không phải để thắng.
- **So với SNN trên Basys 3:** đây là đối chứng gần nhất vì cùng board, cùng xung nhịp. Bản hiện tại nhanh hơn khoảng 16 lần (32 so với 520 µs), nhưng mạng của họ lớn hơn (128 neuron ẩn so với 32), nên khi so cần nói rõ điều này.

Nguồn: FINN, Umuroglu et al., FPGA 2017 · Spiking NN on low-end FPGAs, arXiv 2507.07284. Số "ước tính" là do mình tính từ số phép MAC, chưa đo trên thiết bị thật.

Cấu trúc cần làm

Hệ thống chia làm 2 nửa nối với nhau qua thư mục `gen/` : bạn AI sinh ra trọng số, bạn vi mạch nạp chúng vào ROM trên FPGA. Hai khối tính toán chính (tô màu) dùng chung một module `dense_layer` .

Bạn AI xuất trọng số vào ROM, phần cứng chạy suy luận trên board



kiến trúc hệ thống · 2 phần việc nối qua thư mục gen/

Khối	Người làm	File	Việc cần làm
Huấn luyện + lượng tử	AI	`ai/train_quant_export.py`	Train MLP, chọn scale, xuất ROM
Mô hình tham chiếu int8	AI	hàm `int8_inference()`	Sinh đáp án bit-exact cho testbench
MAC	Vi mạch	`rtl/mac_pe.sv`	Nhân int8 × int8, cộng dồn int32
Lớp fully-connected	Vi mạch	`rtl/dense_layer.sv`	FSM, ROM BRAM, P MAC song song, requant
Lõi tăng tốc	Vi mạch	`rtl/tinyml_accel.sv`	Bộ đệm, nối 2 lớp, argmax
Giao tiếp board	Vi mạch	`rtl/uart.sv`, `rtl/basys3_top.sv`, `constr/basys3.xdc`	UART, LED 7 đoạn, gán chân
Kiểm chứng	Cả hai	`tb/`, `sim/run_sim.sh`	So khớp từng bit phần cứng với Python

Khối	Người làm	File	Việc cần làm
Demo trên PC	AI	`ai/send_image.py`	Gửi ảnh xuống board, đọc kết quả

Phương pháp

Mô hình Python là "đáp án chuẩn": phần cứng phải cho ra đúng từng bit như Python, nhờ vậy mỗi bạn làm độc lập mà vẫn ghép được.

1. Huấn luyện (bạn AI). MLP 784-32-10, ReLU, tối ưu Adam, 15 epoch trên 60.000 ảnh. Đầu vào train là `(pixel >> 1) / 127`, đúng cái phần cứng nhìn thấy.

2. Lượng tử hoá sau huấn luyện (bạn AI). int8 đối xứng, một scale cho cả tensor, không zero-point:

- Trọng số: `W_q = round(W / s_w)`, với `s_w = max|W| / 127`
- Bias: int32, `b_q = round(b / (s_w · s_in))`
- Thang đo lớp ẩn `s_h`: lấy phân vị 99,99 % activation trên 10.000 ảnh train
- Requant lớp ẩn chỉ dùng số nguyên (không cần dấu phẩy động trên FPGA):

```
h  = clamp( (acc × M1 + 2^23) >> 24 , 0, 127 )
M1 = round( s_w1 × s_x / s_h × 2^24 )
```

3. Kiến trúc phần cứng (bạn vi mạch). Luồng dữ liệu kiểu output-stationary: mỗi nhóm P neuron giữ tổng trong P bộ MAC, quét hết các đầu vào. Mỗi chu kỳ đọc 1 byte đầu vào dùng chung cho P MAC và 1 word ROM chứa P trọng số. Sau đó P kết quả đi qua một khối requant dùng chung.

- Số chu kỳ mỗi lớp khoảng (OUT/P) × (IN + P + 3).
- P là núm vặn đánh đổi tài nguyên (DSP, độ rộng BRAM) lấy tốc độ.
- Trọng số nằm trong BRAM trên chip, khởi tạo bằng `$readmemh`.

4. Kiểm chứng (cả hai). Python xuất 200 ảnh test kèm đáp án (nhãn dự đoán + 10 logit). Testbench chạy RTL trên các ảnh đó và so từng bit, rồi kiểm tra thêm cả hệ thống qua UART. Chỉ nạp board khi mô phỏng đã PASS.

5. Đo đạc. Độ chính xác (float32 so với int8), số chu kỳ mỗi ảnh, tài nguyên (LUT, FF, BRAM, DSP), công suất từ Vivado, so với chạy trên CPU.

Hướng thực hiện

Làm theo thứ tự "chạy được trước, tối ưu sau": tuần 2 phải có bản chạy trên board rồi mới mở rộng. Bước 0 đã có sẵn trong thư mục `tinyml-fpga/` (mô phỏng PASS, int8 đạt 96,57 %).

Bước	Tuần	Bạn AI	Bạn vi mạch	Xong khi
0. Khung dự án	đã có	Script train, lượng tử, xuất ROM	RTL lõi, UART, testbench	`run_sim.sh` báo PASS
1. Năm baseline	1	Chạy lại script, hiểu từng bước lượng tử	Chạy mô phỏng, vẽ giản đồ thời gian `dense_layer`	Cả hai giải thích được hợp đồng `gen/`
2. Lên board	2	Viết công cụ gửi ảnh, tiền xử lý ảnh vẽ tay	Tổng hợp Vivado, đạt timing 100 MHz, nạp board	Board nhận đúng ảnh test qua UART
3. Khám phá đánh đổi	3–4	Thử lớp ẩn 16/64/128, huấn luyện có lượng tử (QAT)	Quét P = 4/8/16/32, ghi tài nguyên và số chu kỳ	Bảng độ chính xác và bảng tài nguyên
4. Tối ưu	5–6	Scale theo từng neuron hoặc trọng số 4 bit	Bỏ qua pixel 0, 2 MAC mỗi DSP, đếm chu kỳ trong phần cứng	Số đo trước và sau tối ưu
5. Mở rộng (tuỳ chọn)	7	CNN nhỏ hoặc keyword spotting	Khối tích chập, line buffer	Chỉ làm nếu bước 2–4 xong
6. Báo cáo, demo	8	So sánh với CPU, viết phần AI	Đo công suất, viết phần phần cứng	Demo nhận dạng trực tiếp

Mọi thay đổi quy ước (cách lượng tử, bố cục ROM) phải sửa cả Python lẫn RTL, và chạy lại `run_sim.sh` cho PASS trước khi gộp.

Tiêu chí đánh giá và rủi ro

Nhóm đạt yêu cầu khi board nhận dạng đúng và có số đo cho cả 4 chỉ tiêu dưới đây. Cột "Hiện tại" là số đã đo trên khung có sẵn; tài nguyên và công suất chưa có vì chưa chạy Vivado.

Chỉ tiêu	Mục tiêu	Hiện tại
Độ chính xác int8 (10.000 ảnh test)	≥ 95 %	96,57 %
Khớp giữa phần cứng và Python	100 % từng bit	200/200 ảnh khớp (mô phỏng)
Độ trễ mỗi ảnh @ 100 MHz	≤ 50 µs	32 µs (3.227 chu kỳ, mô phỏng)
Tài nguyên Basys 3	< 50 % DSP và BRAM	cần báo cáo Vivado (ước tính khoảng 25 % DSP, 16 % BRAM)

Rủi ro	Cách giảm
Hai bạn hiểu khác nhau về cách lượng tử	Hợp đồng `gen/` viết rõ trong README, testbench so từng bit
Vivado không suy ra BRAM	Kiểm tra Utilization Report; dự phòng bằng IP Block Memory với file `.coe`
Không đạt timing 100 MHz	Thêm tầng register, hoặc hạ xuống 50 MHz
UART chậm (68 ms mỗi ảnh) che mất tốc độ bộ tăng tốc	Đếm chu kỳ trong phần cứng, hoặc nạp sẵn ảnh vào BRAM để đo
Ảnh vẽ tay thật cho kết quả kém	Tiền xử lý giống MNIST trên PC (căn giữa, co về 20×20)
Ôm đồm CNN/KWS khi baseline chưa chạy	Mốc tuần 2 bắt buộc phải chạy trên board

Code và phân tích chi tiết nằm trong thư mục `tinyml-fpga/` của dự án (`README.md` , `PHAN_TICH.md`).